

AI-authored GUI mental model: start here

Read these small diagrams first, then follow their links to the source-backed code diagrams. The shared Application owns feature meaning, visible state and layout. The host chooses a concrete GUI adapter. Store owns record rules and data. This is a deliberately simple explanation authored by an AI; the local documentation generator reads this static file and needs no AI.

This separately labeled guide contains static AI-authored explanations. The local documentation builder renders it without calling AI. Follow its boxes into the captured code diagrams for implementation details.

AI-AUTHORED__GUI-MENTAL-MODEL.pdf

Authorship: AI-authored conceptual explanation

Authored: 2026-10-04

Source links checked: 24 anchors. The explanation remains separately authored conceptual guidance.

From main to a widget

How does the executable reach the code that creates a widget?

The shared GUI boundary

What stays shared when the GUI backend changes?

Shared widget data becomes a presented view

When does creating a widget become something visible?

An Add click reaches core, then refreshes the view

Which shared functions run when the Add button is clicked?

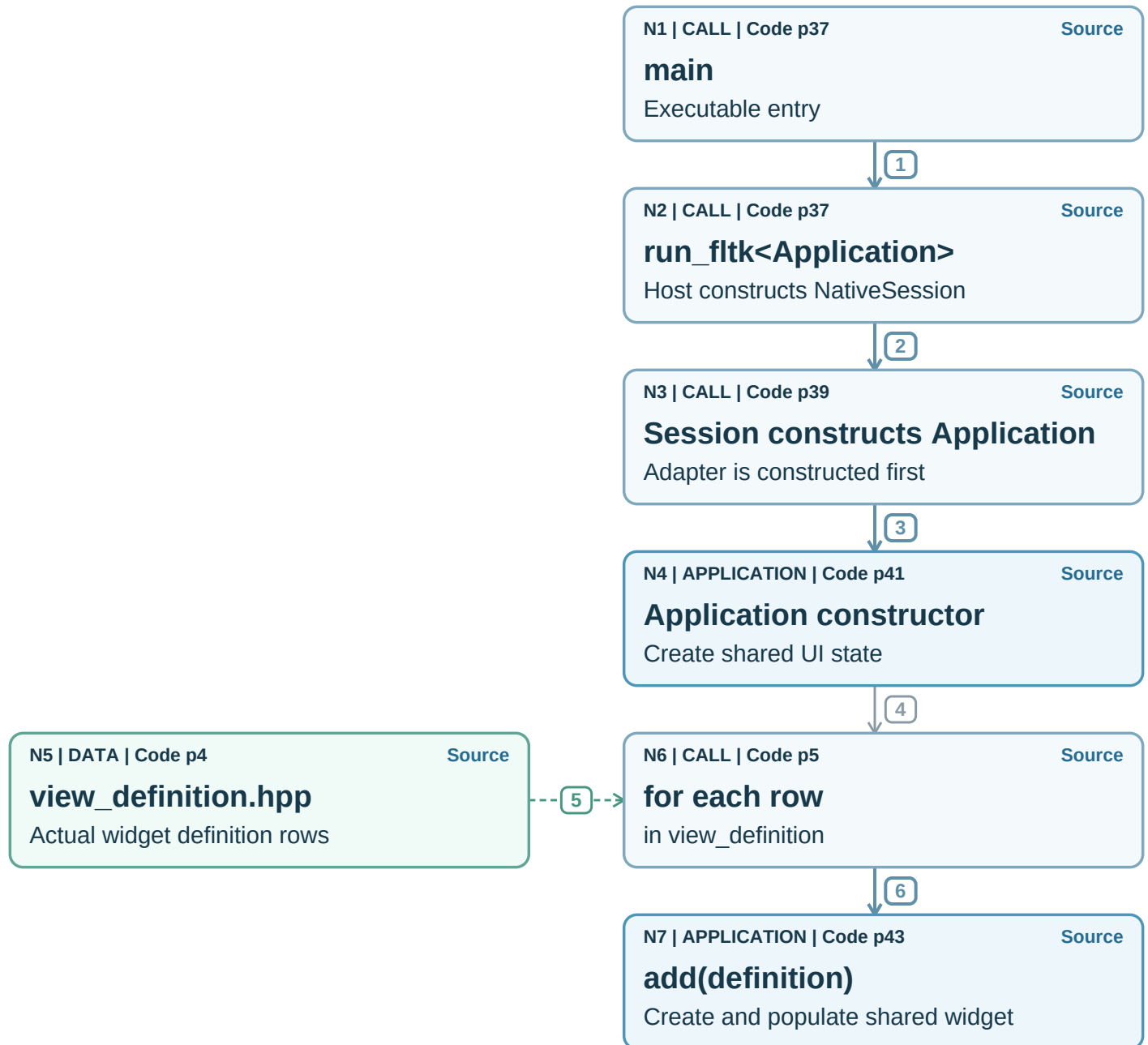
Reading this guide

- These are conceptual paths, not complete execution traces; guards, retries and some helper steps are omitted.
- FLTK illustrates native startup. Other hosts reuse the shared application through the same contract and required capabilities.
- This explanation may lag the code. Regenerate documentation only when explicitly wanted.

From main to a widget

How does the executable reach the code that creates a widget?

Follow the downward call path. The declaration table feeds the loop from the side as data. Application::add creates one shared Snapshot widget per eligible declaration.



1. N1 -> N2 [call] call

3. N3 -> N4 [call] application(adapter)

5. N5 -> N6 [data] supplies data

2. N2 -> N3 [call] construct

4. N4 -> N6 [step] constructor body

6. N6 -> N7 [call] add(definition)

- NativeSession is an alias for Session with native file services.
- Session registers the adapter callback, constructs Application, then enables dispatch through current_.
- The constructor skips the optional Remove declaration; its flag is true.

The shared GUI boundary

What stays shared when the GUI backend changes?

Application sends Snapshot data outward and receives Event input inward through a common Adapter contract. Feature decisions and Store calls remain shared. The host selects the concrete backend that implements the contract.



1. N1 -> N2 [data] publish view
3. N3 -> N4 [step] implementation
5. N5 -> N6 [event] host callback

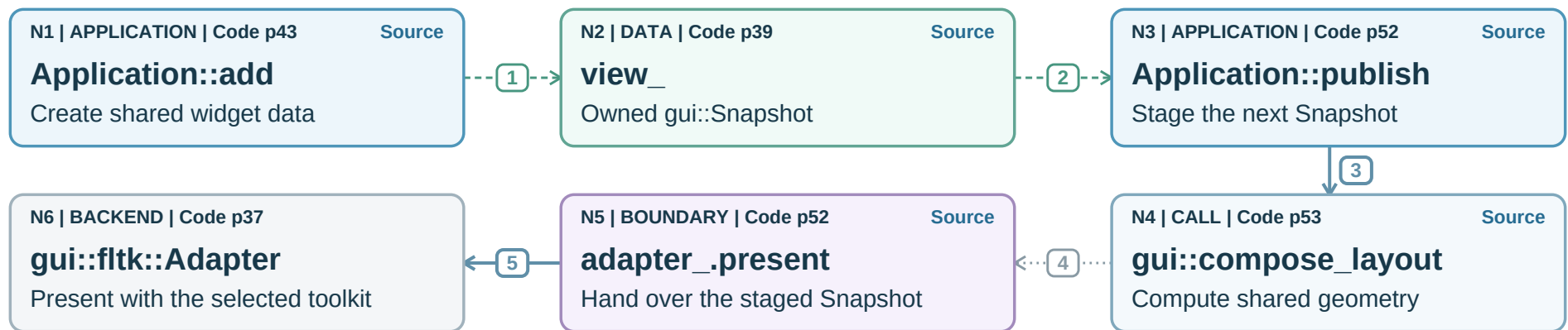
2. N2 -> N3 [data] present(snapshot)
4. N4 -> N5 [event] input
6. N6 -> N7 [call] record actions

- Snapshot and Event are shared data contracts; they render no pixels themselves.
- Application/handle show one application; Adapter/FLTK show one adapter through its interface and implementation.
- Shared controls and handlers stay in Application; supported capabilities are still part of the backend contract.

Shared widget data becomes a presented view

When does creating a widget become something visible?

Application::add fills the owned view model. Application::publish stages the next Snapshot, computes shared layout and asks the adapter to present it. The concrete backend performs toolkit painting.



1. N1 -> N2 [data] append widget
3. N3 -> N4 [call] compose
5. N5 -> N6 [call] concrete implementation

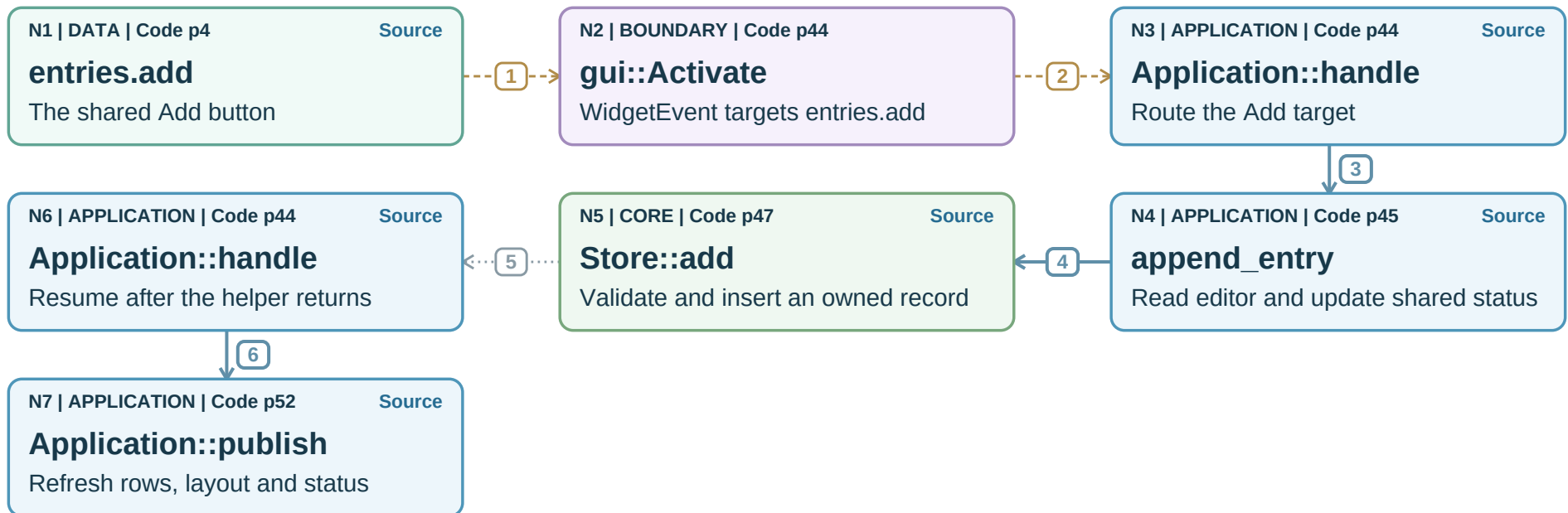
2. N2 -> N3 [data] copy into next
4. N4 -> N5 [return] back in publish

- add creates shared widget data. It does not create a native toolkit control.
- The constructor and event handlers decide when to call publish.
- publish assigns the staged Snapshot to view_ after present returns successfully.

An Add click reaches core, then refreshes the view

Which shared functions run when the Add button is clicked?

The backend reports an activation for `entries.add`. The shared handler routes it to `append_entry`, which calls `Store::add`. Control returns through the helper to the handler, and the handler publishes the resulting state.



1. N1 -> N2 [event] backend input

3. N3 -> N4 [call] `append_entry()`

5. N5 -> N6 [return] via `append_entry`

2. N2 -> N3 [event] host callback

4. N4 -> N5 [call] `entries_.add(text)`

6. N6 -> N7 [call] `publish()`

- `handle` checks and normalizes current input before routing the target ID.

- `Store` returns to `append_entry`; the helper returns to `handle` before publication.

- Expected input/capacity rejection is caught by the helper and published as shared status.